

Reviewer Pack — Minimal Order of Integer Partitions and Resolution Proof Wid...

Entry 990d56439222 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 15:40:07 UTC

Minimal Order of Integer Partitions and Resolution Proof Width Correlation

Entry ID: 990d56439222 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 15:40:07 UTC

1. Conjecture

Field A (mathematical branch): Partition Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the minimal order of an integer partition (denoted as $\text{min_order}(\varphi)$) is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{min_order}(\varphi) = \Theta(w(\varphi))$. Additionally, for all instances with at most k distinct variables, $\text{min_order}(\varphi) = O(k)$.

Rationale (proposer's reasoning):

Partition theory deals with the ways of breaking down a set into non-empty subsets. This conjecture explores the relationship between the structure of these partitions and the complexity of resolving the CNF, which may expose previously unknown connections between combinatorial structures and computational hardness.

Taxonomy category: partition_theory_resolution_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ecd79161cd85fd6f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given CNF φ with n variables, if $|\text{Pearson correlation coefficient}| \geq 0.7$ and $\text{min_order}(\varphi) / w(\varphi) \leq 1.5$, then support the conjecture; otherwise, falsify it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Minimal Order Integer Partitions AND Resolution Proof Width - min_order(CNF) AND resolution proof width-Partition Theory AND Resolution Proof Complexity AND integer partition

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_cnf(n: int, m: int) -> list:
    variables = list(range(1, n + 1))
    cnf = []
    for _ in range(m):
        k = random.randint(1, n)
        clause = random.sample(variables, k)
        cnf.append(clause)
    return cnf

def min_order(cnf: list) -> int:
    from itertools import combinations
    def is_partition(partition):
        for subset in partition:
            if not any(all(abs(x) == abs(y) for x, y in zip(subset, subset[1:])) and len(set(abs(x) for x in subset)) == len(subset)):
                return False
        return True

    n = len(cnf)
    min_order = float('inf')
    for r in range(1, n + 1):
        for partition in combinations(range(n), r):
            if is_partition(partition):
                min_order = min(min_order, r)
    return min_order

def resolution_width(cnf: list) -> int:
    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit_clauses = [c for c in clauses if len(c) == 1]
```

```

        if not unit_clauses:
            return False
        literal = unit_clauses[0][0]
        new_assignment = assignment.copy()
        new_assignment[abs(literal)] = literal > 0
        if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
            return True
        new_assignment[abs(literal)] = not literal > 0
        return dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment)

assignment = {}
return len(cnf) - sum(dpll(cnf, assignment) for _ in range(10))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            cnf = generate_cnf(n, random.randint(n, 2 * n))
            instances_tested += 1
            min_order_val = min_order(cnf)
            width_val = resolution_width(cnf)
            metric_values.append(min_order_val / width_val)

    if len(metric_values) < 30:
        return {
            "metric_name": "min_order_over_resolution_width",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    correlation_coefficient = sum((x - mean) * (y - mean) for x, y in zip(metric_values, [sum(metric_values)]))
    if abs(correlation_coefficient) < 0.7 or max(metric_values) > 1.5:
        conjecture_holds = False
        counterexample = "correlation_threshold_not_met"

    return {
        "metric_name": "min_order_over_resolution_width",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_threshold_not_met\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data_or_unsupported")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7d86512.py", line 104, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7d86512.py", line 71, in run_trial
    min_order_val = min_order(cnf)
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7d86512.py", line 37, in min_order
    if is_partition(partition):
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7d86512.py", line 29, in is_partition
    if not any(all(abs(x) == abs(y) for x, y in zip(subset, subset[1:]))) and len(set(abs(x) for x in subset)) > 1:
                                     ~~~~~^
TypeError: 'int' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and the ratio of min_order to resolution proof width. | next: Investigate the cause of the crash in the test code and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14295
2	propose	ollama_remote	glm4:latest	0	0	16018
3	preregistration	ollama_remote	glm4:latest	0	0	13673
4	novelty	ollama_remote	glm4:latest	0	0	15870
5	novelty	ollama_remote	glm4:latest	0	0	8974
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25039
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18125
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16011
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15197
10	judge	ollama_remote	glm4:latest	0	0	27161

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 170363 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/990d56439222.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/990d56439222.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/990d56439222.tar.gz` (if generated)